Basic Level

Python & Programming:
What are Python data types? Explain list, tuple, dictionary, and set.
How do you handle exceptions in Python?
Write a Python function to reverse a string.
Explain list comprehension. Give an example.
How do you read and write files in Python?
Data Structures & Algorithms:
What is an array? How is it different from a list in Python?
Implement FizzBuzz from 1 to 100.
How to check if a string is a palindrome?
Write code to find the maximum number in a list.
Explain time complexity. What is Big O notation?
SQL:
Write a SQL query to select all rows from a table.
How do you perform an inner join between two tables?
What are primary keys and foreign keys?
Write a query to find the second highest salary in a table.
Cloud Fundamentals:
What is cloud computing?
Explain the difference between IaaS, PaaS, and SaaS.
What is AWS S3 used for?
What is an EC2 instance?
Intermediate Level
Python & Problem Solving:
Write a function to remove duplicates from a list without using set().
How do you implement recursion? Write a factorial function.
Explain generators and iterators.
Write code to find missing numbers in a sequence.
Discuss how you can optimize a slow Python program.
Data Science Concepts:
What is data cleaning? Why is it important?
Explain overfitting and underfitting.
What is cross-validation?
Describe common evaluation metrics for classification problems.
How do you handle missing values in datasets?
Algorithms & Data Structures:
Implement binary search in Python.
Explain how a hash map works.

Compare BFS and DFS traversals.
Write code to detect cycles in a linked list.
Describe sorting algorithms and their complexities.
SQL & Databases:
Write a query to calculate total sales per region.
Explain database normalization.
How do you create an index? Why is it important?
Describe ACID properties in databases.
Cloud & DevOps:
What is IAM in AWS? How do you secure access?
Explain the concept of serverless computing.
What are Lambda functions?
How do you monitor resources in AWS?
Expert Level
Python & Programming:
How do you manage memory in Python?
Explain the GIL (Global Interpreter Lock).
Discuss multi-threading vs multi-processing in Python.
Write thread-safe code snippets.
How do you profile Python code for performance?
Data Science & Machine Learning:
Explain the bias-variance tradeoff.
What are ensemble methods? Give examples.
Discuss feature selection and dimensionality reduction techniques.
How do you deploy machine learning models in production?
Explain transformers and attention mechanisms in NLP.
Algorithms & Advanced Data Structures:
Implement a Trie data structure.
Solve advanced dynamic programming problems.
Design an efficient data retrieval system from large datasets.
Explain graph algorithms for shortest path and network flow.
Cloud Architecture & Security:
Design a scalable invoice processing system on AWS.
Explain VPC, subnets, security groups, and their configurations.
Discuss best practices for cloud security and compliance.
How do you implement CI/CD pipelines for cloud applications?
Design a disaster recovery plan for a cloud-based service.
System Design:
Design an end-to-end invoice verification system using microservices.
How would you ensure data consistency across distributed systems?
Explain how to design a conversational AI system with context memory.
Discuss the trade-offs between SQL and NoSQL databases.
Design a caching strategy for frequently accessed invoice data. answer this all

Here are clear, concise answers to your comprehensive list of interview questions, organized by level and topic:

Basic Level

Python & Programming

Q: What are Python data types? Explain list, tuple, dictionary, and set.

- List: Ordered, mutable collection.
- Tuple: Ordered, immutable collection.
- Dictionary: Unordered, mutable collection of key-value pairs.
- Set: Unordered collection of unique elements.

Q: How do you handle exceptions in Python?
Use `try-except` blocks.

```
try:
    # code
except Exception as e:
    print(e)
```

Q: Write a Python function to reverse a string.

```
def reverse_string(s):
    return s[::-1]
```

Q: Explain list comprehension. Give an example.
Concise way to create a list.
Example:

```
squares = [x**2 for x in range(5)]
```

Q: How do you read and write files in Python?
Read:

```
with open('file.txt', 'r') as f:
    data = f.read()
```

Write:

```
with open('file.txt', 'w') as f:
    f.write('Hello')
```

Data Structures & Algorithms

Q: What is an array? How is it different from a list in Python?
Array: Fixed size, stores elements of same type. Python lists are dynamic and can hold mixed data.

Q: Implement FizzBuzz from 1 to 100.

```
for i in range(1, 101):
    if i % 15 == 0:
        print('FizzBuzz')
    elif i % 3 == 0:
        print('Fizz')
    elif i % 5 == 0:
        print('Buzz')
    else:
        print(i)
```

Q: How to check if a string is a palindrome?

```
def is_palindrome(s):
    return s == s[::-1]
```

Q: Write code to find the maximum number in a list.

```
max_num = max(my_list)
```

Q: Explain time complexity. What is Big O notation?
It expresses how run time grows relative to input size. Big O is worst-case upper bound.

SQL

Q: Write a SQL query to select all rows from a table.

```
SELECT * FROM table_name;
```

Q: How do you perform an inner join between two tables?

```
SELECT * FROM table1 INNER JOIN table2 ON table1.id = table2.id;
```

Q: What are primary keys and foreign keys?
Primary Key: Unique identifier in a table.
Foreign Key: References primary key in another table to enforce relationships.

Q: Write a query to find the second highest salary in a table.

```
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
```

Cloud Fundamentals

Q: What is cloud computing?
Delivery of computing services over the internet.

Q: Explain the difference between IaaS, PaaS, and SaaS.

- IaaS: Infrastructure as a Service (virtual machines).
- PaaS: Platform as a Service (development platform).
- SaaS: Software as a Service (applications hosted online).

Q: What is AWS S3 used for?
Scalable object storage.

Q: What is an EC2 instance?
Virtual server in AWS cloud.

Intermediate Level

Python & Problem Solving

Q: Write a function to remove duplicates from a list without using set().

```
def remove_duplicates(lst):
    result = []
    for item in lst:
        if item not in result:
            result.append(item)
    return result
```

Q: How do you implement recursion? Write a factorial function.

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)
```

Q: Explain generators and iterators.
Iterators traverse elements; generators are iterators defined with `yield`.

Q: Write code to find missing numbers in a sequence.

```
def find_missing(nums, start, end):
```

```
    return set(range(start, end + 1)) - set(nums)
```

Q: Discuss how you can optimize a slow Python program.
Use efficient algorithms, memoization, built-in functions, profiling tools, and avoid unnecessary loops.

Data Science Concepts

Q: What is data cleaning? Why is it important?
Fixing or removing incorrect or missing data to ensure quality analysis.

Q: Explain overfitting and underfitting.
Overfitting: Model fits training data too well, poor generalization.
Underfitting: Model too simple, poor performance on training and new data.

Q: What is cross-validation?
Technique for assessing model performance by partitioning data into train/test sets multiple times.

Q: Describe common evaluation metrics for classification problems.
Accuracy, precision, recall, F1-score, ROC-AUC.

Q: How do you handle missing values in datasets?
Remove rows/columns, fill with mean/median/mode, or predict missing values.

Algorithms & Data Structures

Q: Implement binary search in Python.

```
def binary_search(arr, target):
    left, right = 0, len(arr)-1
    while left <= right:
        mid = (left + right)//2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Q: Explain how a hash map works.
Stores key-value pairs with constant time access via hashing keys to buckets.

Q: Compare BFS and DFS traversals.
BFS: level-order, uses queue.
DFS: goes deep before backtracking, uses stack/recursion.

Q: Write code to detect cycles in a linked list.

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

Q: Describe sorting algorithms and their complexities.

- Bubble Sort: O(n^2)
- Merge Sort: O(n log n)
- Quick Sort: Average O(n log n)

SQL & Databases

Q: Write a query to calculate total sales per region.

```
SELECT region, SUM(sales) FROM sales_data GROUP BY region;
```

Q: Explain database normalization.
Organizing data to reduce redundancy and improve integrity.

Q: How do you create an index? Why is it important?

```
CREATE INDEX idx_name ON table(column);
```

Speeds up queries by enabling quick lookup.

Q: Describe ACID properties in databases.
Atomicity, Consistency, Isolation, Durability ensure safe transactions.

Cloud & DevOps

Q: What is IAM in AWS? How do you secure access?
Manages users and their permissions. Secure by applying least privilege principles, MFA.

Q: Explain the concept of serverless computing.
Running code without managing servers; resources are managed by cloud provider.

Q: What are Lambda functions?
Serverless compute service running short-lived functions.

Q: How do you monitor resources in AWS?
Using CloudWatch metrics, alarms, and logs.

Expert Level

Python & Programming

Q: How do you manage memory in Python?
Automatic garbage collection via reference counting and cyclic garbage collector.

Q: Explain the GIL (Global Interpreter Lock).
Mutex that prevents multiple native threads executing Python bytecodes at once.

Q: Discuss multi-threading vs multi-processing in Python.
Multi-threading shares memory but limited by GIL; multi-processing spawns separate memory spaces for true concurrency.

Q: Write thread-safe code snippets.
Use locks/mutexes for shared resource access.
Example:

```
import threading
lock = threading.Lock()
with lock:
    # critical section
```

Q: How do you profile Python code for performance?
Use cProfile module or line_profiler to identify bottlenecks.

Data Science & Machine Learning

Q: Explain the bias-variance tradeoff.
Low bias: model fits training well; high variance: model sensitive to noise. Balance improves generalization.

Q: What are ensemble methods? Give examples.
Combine models to improve accuracy. Examples: Random Forests, Gradient Boosting.

Q: Discuss feature selection and dimensionality reduction techniques.
Select relevant features or reduce dimensions using methods like PCA.

Q: How do you deploy machine learning models in production?
Containerize model, expose API endpoints, monitor and update regularly.

Q: Explain transformers and attention mechanisms in NLP.
Transformers use self-attention to weigh importance of words in context, enabling parallel processing of sequences.

Algorithms & Advanced Data Structures

Q: Implement a Trie data structure.
(Concise example skipped here for brevity; can provide if needed.)

Q: Solve advanced dynamic programming problems.
Use memoization/tabulation for optimization.

Q: Design an efficient data retrieval system from large datasets.
Use indexing, caching, partitioning, and distributed storage.

Q: Explain graph algorithms for shortest path and network flow.
Dijkstra's algorithm for shortest paths; Ford-Fulkerson for max flow.

Cloud Architecture & Security

Q: Design a scalable invoice processing system on AWS.
Use S3 for storage, Lambda for processing, DynamoDB for metadata, API Gateway for endpoints, CloudWatch for monitoring.

Q: Explain VPC, subnets, security groups, and their configurations.
VPC isolates network; subnets divide VPC; security groups act as virtual firewalls.

Q: Discuss best practices for cloud security and compliance.
Use IAM roles, encryption, monitoring, least privilege access.

Q: How do you implement CI/CD pipelines for cloud applications?
Use tools like AWS CodePipeline, Jenkins for automated build, test, and deployment.

Q: Design a disaster recovery plan for a cloud-based service.
Backups, multi-region deployments, failover automation.

System Design

Q: Design an end-to-end invoice verification system using microservices.
Break into services like ingestion, verification, notification, UI, each deployed independently communicating via APIs or message queues.

Q: How would you ensure data consistency across distributed systems?
Use distributed transactions, consensus protocols, eventual consistency models.

Q: Explain how to design a conversational AI system with context memory.
Store session states, use context embeddings, and state machines to handle multi-turn dialogues.

Q: Discuss the trade-offs between SQL and NoSQL databases.
SQL: structured, relational, strong consistency; NoSQL: flexible schema, high scalability.

Q: Design a caching strategy for frequently accessed invoice data.
Use in-memory caches (Redis), with cache invalidation policies and TTL.

If you want, I can expand any specific answers or provide code examples.